

Mobile Applications Development

Lisa Giacobazzi

2025

Contents

1	Informazioni generali	7
1.1	Obiettivi e motivazioni	7
1.2	Progettazione Mobile	7
1.2.1	Eterogeneità	7
1.2.2	Tu non sei l'utente	7
1.3	Competenze richieste	7
1.4	Tecnologie di sviluppo mobile nel 2025	8
1.5	Sviluppo nativo	8
1.5.1	Cross-platform	8
1.5.2	Web e Low-code	8
1.6	Contenuti del corso	8
1.7	Metodo: Design Thinking e Agile	8
1.8	Valutazione	8
1.8.1	Peer Score	8
1.9	Organizzazione	8
1.10	Progetto finale	9
1.11	Primo assignment	9
1.12	Uso dei LLM	9
2	Introduzione al linguaggio Kotlin	10
2.1	Sistema di tipi	10
2.2	Unit, Nothing, Any	10
2.3	Variabili e Valori	10
2.4	Funzioni	11
2.5	Classi e oggetti	11
2.6	Classi avanzate	11
2.7	Ereditarietà	12
2.8	Generics e varianti	12
2.9	Funzioni Scoped	12
2.10	Collections e Sequences	12
2.11	Casting e Smart Casts	13
3	Introduzione a Gradle	14
3.1	Principi generali	14
3.2	Struttura e script di build	14
3.3	Inizializzare un progetto Gradle	14
3.4	Struttura di un progetto Gradle	14
3.5	Multi-modulo	14
3.6	File di configurazione	15
3.6.1	Parent build.gradle	15
3.6.2	Modulo build.gradle	15
3.6.3	settings.gradle.kts	15
3.7	SDK Android	15
3.8	Tasks	15
3.9	Plugin Gradle	15
3.10	Dipendenze e Repositories	15
3.10.1	Dipendenze transitiva	16
3.10.2	Configurazione	16
4	Introduzione	17

4.1	L'Open Handset Alliance (OHA)	17
4.2	Architettura di Android	17
4.3	Sicurezza in Android	17
4.4	Sviluppo di applicazioni Android	17
4.5	Struttura delle applicazioni	17
4.6	Dettagli sui componenti	18
4.6.1	Activity	18
4.6.2	Service	18
4.6.3	Content Provider	18
4.6.4	Broadcast Receiver	18
4.7	Ciclo di vita delle applicazioni	18
4.7.1	Manifest e avvio	18
4.7.2	Oggetto Application	18
4.7.3	Persistenza	18
4.8	Avvio di un'applicazione	18
4.9	Interazione tra applicazioni	19
5	Componenti delle App Android	20
5.1	La classe Activity	20
5.1.1	Responsabilità di un'Activity	20
5.2	Ciclo di vita e Stack delle Activity	20
5.2.1	Gestione corretta del ciclo di vita	20
5.2.2	Salvataggio dello stato	20
5.3	Preparazione dell'interfaccia grafica	21
5.4	Approccio imperativo	21
5.4.1	Approccio dichiarativo con XML	21
5.4.2	Jetpack Compose	21
5.5	Intents	21
5.6	Transizioni tra Activity	21
5.7	Broadcast degli Intent	21
5.8	La classe Context	21
6	Introduzione a Jetpack Compose	22
6.1	Principi fondamentali	22
6.2	Sintassi e DSL	22
6.3	Elementi base	22
6.4	Layout base	22
6.5	Liste e griglie	22
6.6	Modifiers	23
6.7	Gestione dello stato	23
6.8	Hoisting e flusso dati	23
6.9	Stilizzazione con Material Theme	23
7	Concurrency e threading in Android	24
7.1	Thread in Kotlin	24
7.1.1	Problemi comuni	24
7.2	Multithreading in Android	24
7.2.1	Looper e MessageQueue	24
7.3	Thread di sistema e JVM	24
7.4	Astrazioni di programmazione	24
7.5	Uso di Handler e Looper	24
7.6	HandlerThread	25

7.7	Problemi tipici e best practice	25
7.8	Producer-consumer pattern	25
8	Introduzione alle Coroutines	26
8.1	Cos'è una Coroutine	26
8.2	Motivazione e vantaggi	26
8.3	Suspend function e CoroutineScope	26
8.4	Creazione di Coroutines	26
8.5	Structured Concurrency	26
8.6	Dispatcher e Context	26
8.7	Exception e Cancellation	26
8.8	Sincronizzazione e concorrenza	27
8.9	Flows	27
8.9.1	Operazioni intermedie e terminali	27
8.10	Flows vs Sequence	27
9	Architettura delle applicazioni	28
9.1	Pattern architetturali	28
9.2	Architettura delle app Android	28
9.3	Separation of Concerns	28
9.4	Lo stato dell'applicazione	28
9.5	Principali architetture	28
9.6	MVC - Model View Controller	28
9.6.1	MVP - Model View Presenter	29
9.6.2	MVI - Model View Intent	29
9.6.3	MVVM - Model View ViewModel	29
9.7	Android e MVVM	29
9.8	ViewModel: caratteristiche	29
9.9	Osservabili e Compose	29
9.10	Esempi ViewModel	29
9.11	Lifecycle-aware components	30
9.12	Conclusione	30
10	Introduzione alla Data Layer	31
10.1	Modelli e disaccoppiamento	31
10.2	Aggiornamenti asincroni e stato	31
10.3	Operazioni lunghe e threading	31
10.4	Architettura del Model	31
10.5	Implementazione del Model	32
10.6	Persistenza locale con Room	32
10.7	DAO e entità	32
10.8	Database singleton	32
10.9	Accesso ai dati	33
10.10	Accesso remoto con Retrofit	33
10.11	Retrofit + Flow	33
10.12	Conclusioni	33
10.13	Introduzione alla Dependency Injection	34
10.14	Problemi nella gestione delle dipendenze	34
10.15	Principio di Inversion of Control e DI	34
10.16	Refactoring e interfacce	34
10.17	Esempio	34
10.18	Framework per la DI in Android	34

10.19	Introduzione a Hilt	34
10.20	Configurazione	35
10.21	Annotazioni principali	35
10.22	Scope	35
10.23	Esempio con Hilt	35
10.24	Modules con Hilt	35
10.25	Integrazione con Jetpack	36
11	Introduzione alla Navigazione	37
11.1	Dipendenze necessarie	37
11.2	Componenti principali della Navigation	37
11.3	Definizione delle route	37
11.4	Navigazione tra schermate	37
11.5	Gestione dello stato di navigazione	37
11.6	Scaffold e Navigation UI	38
11.7	Altri componenti di navigazione	38
12	Input dell'utente nei sistemi Android	39
12.0.1	MotionEvent	39
12.1	Gestione dei tocchi in Compose	39
12.1.1	Esempio: tap e long press	39
12.2	Drag e Swipe	39
12.3	Gesti multitouch	39
12.4	Animazioni: introduzione	39
12.5	12 principi dell'animazione secondo Disney	40
12.6	Animazioni in Compose	40
12.6.1	Cambi di gerarchia	40
12.6.2	Animazioni di attributi	40
12.6.3	Easing	41
13	Introduzione a Firestore	42
13.1	Modello NoSQL e Architettura	42
13.2	Querying e Sincronizzazione	42
13.3	Modellazione dei dati	42
13.4	Relazioni tra dati	42
13.5	Accesso da Android	42
13.6	Scrittura e Lettura dei dati	42
13.7	Scrittura	42
13.7.1	Lettura	43
13.8	Kotlin e serializzazione	43
13.9	Realtime updates e Flow	43
13.10	Query avanzate	43
13.11	MVVM e Firestore	43
13.12	Sicurezza e regole di accesso	43
13.13	Operazioni atomiche	43
13.14	Persistenza offline e indexing	43
14	Introduzione ai Services	44
14.1	Started Services	44
14.2	Bound Services	44
14.3	Gestione del ciclo di vita dei servizi	44
14.4	Implementare un servizio personalizzato	44

14.4.1	onStartCommand e ritorni	44
14.5	Limitazioni e Foreground Services	44
14.6	Alternative moderne: WorkManager	45
14.6.1	Esempio di Worker	45
14.7	Bound Services con IBinder	45
14.7.1	Esempio con Binder	45
14.8	Dichiarazione nel Manifest	45
15	Panoramica dello sviluppo cross-platform	46
15.1	Motivazioni principali	46
15.2	Limitazioni e problematiche	46
15.3	Differenze strutturali tra Android e iOS	46
15.4	Framework e tecnologie principali	46
15.5	WebView-based	46
15.5.1	Native-widget-based	46
15.5.2	Custom-widget-based	46
15.6	Kotlin Multiplatform (KMP)	47
15.7	Compose Multiplatform	47
15.8	Confronto tra tecnologie	47
15.9	Tendenze emergenti	47
16	XR: Extended Reality	48
16.0.1	Virtual Reality (VR)	48
16.0.2	Immersione e Presenza	48
16.1	AR e MR	48
16.2	XR System: QOC e Interaction	48
16.3	Tecniche di Manipolazione	48
16.4	Locomotion in VR	48
16.5	Dispositivi XR	48
16.6	Display e UI in XR	48
16.7	AndroidXR: OS e Visione	49
16.8	Jetpack Compose XR: SpatialUI	49
16.9	Jetpack SceneCore	49
16.10	HSM vs FSM	49
16.11	Conclusione	49

1 Informazioni generali

Il corso di **Mobile Application Development** per l'anno accademico 2024–2025 è tenuto dal professor **F. Gabriele Praticò**, del Dipartimento di Automatica e Informatica (DAUIN) del Politecnico di Torino.

Le aree di ricerca del docente includono:

- Interazione uomo-macchina (HCI)
- Realtà aumentata/virtuale (XR, VR/AR)
- Sistemi di simulazione e training
- Interfacce per veicoli autonomi (HMI)

Contatti:

- gabriele.prattico@polito.it
- Siti web: <http://grains.polito.it/work.php>, <https://vr.polito.it>

Collaboratori: Alessandro Russo, Lorenzo Valente

1.1 Obiettivi e motivazioni

Nel mondo reale i prodotti e i servizi sono fatti di hardware e software tangibili, ma nella mente del consumatore contano soprattutto le **esperienze d'uso** (UX). Un'interfaccia che non si comporta come l'utente si aspetta può frustrare, anche se il compito viene portato a termine.

Statisticamente:

- Il 25% delle app viene aperto una sola volta e poi abbandonato.
- Il 48% delle app viene usato solo 10 volte o meno.

1.2 Progettazione Mobile

La progettazione di un'app richiede l'analisi del **contesto d'uso**:

- Mobilità, attenzione ridotta, situazioni di stress.
- Interazioni comode e tipiche: gesti touch, regola del pollice, pattern comuni.
- Compiti semplici e pensati per il mobile: display piccoli, qualità connessione, batteria.

1.2.1 Eterogeneità

Bisogna considerare la varietà:

- Di dispositivi (dimensioni, capacità, sistema operativo).
- Di utenti (diverse competenze, preferenze e necessità).

1.2.2 Tu non sei l'utente

Spesso gli sviluppatori non coincidono con gli utenti target: bisogna progettare **per gli altri**.

1.3 Competenze richieste

Lo sviluppo mobile è multidisciplinare:

- Informatica: analisi problemi, architettura software, programmazione, testing.
- Scienze umane: psicologia, fisiologia, sociologia.
- Organizzazione: project management, leadership, divisione task.

1.4 Tecnologie di sviluppo mobile nel 2025

1.5 Sviluppo nativo

- Kotlin + Jetpack Compose (Android)
- Swift + SwiftUI (iOS)
- C++ per SDK nativi a basso livello

1.5.1 Cross-platform

- Kotlin Multiplatform (KMP)
- Flutter (Dart)
- React Native (JavaScript/TypeScript)
- MAUI (.Net)

1.5.2 Web e Low-code

- Progressive Web App (PWA), Capacitor/Ionic
- Soluzioni low/no-code: FlutterFlow, Adalo, Bubble

1.6 Contenuti del corso

- Kotlin
- Framework Android
- User-Centered Design
- Team working

1.7 Metodo: Design Thinking e Agile

- **Empatia con l'utente**, analisi casi d'uso.
- Progettazione: architettura informativa, design visivo/interattivo.
- Solo dopo: sviluppo.
- Agile: obiettivi, test, GitHub, Slack, revisioni settimanali.

1.8 Valutazione

- Esame scritto: max 18 pt, soglia 10 pt (3–5 domande aperte).
- Laboratori: max 10 pt (5 lab, 2 pt ciascuno).
- Progetto finale: max 5 pt.

1.8.1 Peer Score

Punteggio finale del laboratorio:

$$AS = \frac{1}{2} \cdot \text{prof}_{\text{Grade}} + \frac{1}{2} \cdot \text{prof}_{\text{Grade}} \cdot \frac{\text{clamp}_{10}(\text{peerScore})}{10}$$

Con pesi: preparazione (0.25), partecipazione (0.25), contributo (0.5)

1.9 Organizzazione

- Laboratori in gruppi da 4 persone, con almeno 1 smartphone Android.
- Setup del gruppo entro il 7 marzo 2024.
- Modulo Google per segnalazione: <https://forms.gle/DFMy6ufUZeaAjyxUA>
- Foglio collaborativo per trovare membri: <https://docs.google.com/spreadsheets/d/>
- ...

1.10 Progetto finale

App per proporre viaggi e trovare compagni:

- Registrazione e profilo
- Ricerca proposte per luogo, prezzo, durata, tipo
- Candidature, pubblicazione viaggi, recensioni
- Notifiche su proposte, viaggi raccomandati, aggiornamenti

1.11 Primo assignment

Competitive analysis di app simili (es. Piratinviaggio, Airbnb, TripAdvisor): funzionalità, itinerari, UX mobile, mancanze, layout, navigazione.

1.12 Uso dei LLM

- Da usare come tutor, non come scorciatoia
- Utile per test personalizzati, revisione, spiegazioni
- Complemento, non sostituto, delle fonti ufficiali (slides, libri, documentazione)
- <https://lmarena.ai/> – benchmark LLM

2 Introduzione al linguaggio Kotlin

Kotlin è un linguaggio di programmazione moderno, **statisticamente tipizzato** e pensato per la **programmazione multiplatforma**. Nato nel 2011 e mantenuto da JetBrains, combina elementi di programmazione **funzionale** e **object-oriented**, mirando a semplificare lo sviluppo rispetto a Java.

2.1 Sistema di tipi

Kotlin possiede un sistema di tipi moderno, progettato per migliorare la sicurezza a tempo di compilazione e ridurre il numero di errori comuni durante lo sviluppo, in particolare i famigerati null pointer exceptions (NPE).

Le principali caratteristiche del sistema di tipi di Kotlin sono:

- Tipi primitivi non sono distinti da quelli oggetto:
A differenza di Java, dove esiste una distinzione tra tipi primitivi (come `int`, `char`, `boolean`) e le rispettive classi wrapper (`Integer`, `Character`, `Boolean`), in Kotlin ogni tipo è rappresentato da una classe. Ad esempio, `Int`, `Boolean`, `Double` sono classi a tutti gli effetti. Ciò consente a ogni valore, anche numerico, di essere trattato come un oggetto, mantenendo comunque ottime performance grazie all'autoboxing automatico durante la compilazione.
- **Null safety**: distinzione chiara tra tipi nullable (es. `String?`) e non-nullable (es. `String`).
- Operatori speciali per la gestione del null:
 - `?.` (safe call): permette di accedere a una proprietà o chiamare un metodo solo se il valore non è null, altrimenti restituisce null.

```
val length = myString?.length // Int? se myString è nullable
```
 - `?:` (elvis): consente di fornire un valore di fallback in caso il risultato a sinistra sia null.

```
val length = myString?.length ?: 0
```
 - `!!` (assert non null): forza l'interprete a trattare il valore come non-null, generando una `NullPointerException` in caso contrario.

```
val length = myString!!.length
```

2.2 Unit, Nothing, Any

Nel sistema di tipi di Kotlin esistono alcuni tipi speciali che svolgono un ruolo fondamentale nella modellazione dei comportamenti delle funzioni e nella gestione della generalità e dell'eccezione.

- **Unit**
In Kotlin, `Unit` è il tipo di ritorno predefinito per una funzione che non restituisce un valore significativo.
È simile a `void`, ma è una classe con una sola istanza.
- **Nothing** rappresenta l'estremo inferiore: non ha istanze e indica codice che non termina normalmente (es. eccezioni).
- **Any** è il supertipo di tutti i tipi non-nullabili.

2.3 Variabili e Valori

- **var** definisce una variabile riassegnabile.
- **val** definisce una costante (binding immutabile, ma contenuto mutabile se riferimento a oggetti).

2.4 Funzioni

In Kotlin, le funzioni sono cittadini di prima classe: ciò significa che possono essere dichiarate, passate, assegnate e invocate proprio come qualunque altra variabile o oggetto. Questo rende il linguaggio molto espressivo e adatto a paradigmi sia imperativi che funzionali.

- Definite con **fun**. Possono avere parametri di default e nomi dei parametri.
- Possono essere assegnate a variabili (funzioni come tipi).
- Supporto completo per lambda expressions, trailing lambdas, closures.

2.5 Classi e oggetti

In Kotlin, le classi rappresentano il cuore della programmazione orientata agli oggetti, ma con una sintassi compatta e moderna. Offrono molte funzionalità avanzate che riducono il boilerplate e migliorano la leggibilità del codice.

- Sintassi compatta per costruttori primari e secondari.

```
class User(val name: String, var age: Int)
```
- Le proprietà possono essere dichiarate con **val** (immutabile) o **var** (mutabile). Ogni proprietà può avere getter e setter personalizzati, se necessario.
- Modificatori di visibilità: **public**, **private**, **protected**, **internal**.
- Inizializzazione itardata
lateinit permette di dichiarare una var non nullable senza inicializzarla subito, a patto che venga inicializzata prima dell'uso. Usabile solo con tipi non primitivi.
by lazy inicializza una val al primo accesso, ideale per calcoli costosi o inicializzazioni lente.
- Delegati di proprietà:
Kotlin consente di delegare la gestione delle proprietà tramite keyword by.
observable: esegue un blocco ogni volta che il valore cambia;
vetoable simile a observable, ma può impedire il cambiamento restituendo false;
custom.

```
var name: String by Delegates.observable("Lisa") { _, old, new ->
    println("\$old → \$new") }
```

2.6 Classi avanzate

Oltre alle classi tradizionali, Kotlin fornisce diverse forme specializzate di classi che permettono di modellare in modo più espressivo e compatto molti pattern comuni nella programmazione.

- **Data class**: Le data class sono progettate per rappresentare dati “immutabili” o entità di dominio. Specificando data nella dichiarazione, Kotlin genera automaticamente: equals(), hashCode(), toString(), copy(), componenti per la decomposizione component1().
- **Enum class**: set limitati di istanze. Ogni valore è un oggetto singleton e può avere proprietà e metodi.
- **Sealed class**: simili agli enum, ma per sottotipi (ADT).

```
sealed class Result
data class Success(val data: String): Result()
data class Error(val message: String): Result()
```
- **Object** definisce un singleton, ovvero una classe con una sola istanza. È utile per contenere oggetti globali o helper.
companion object per singleton e metodi statici.
- **Extension functions** permettono di “aggiungere” metodi a classi già esistenti senza modificarne il codice originale o ereditarle.

2.7 Ereditarietà

In Kotlin, le classi sono **final** per impostazione predefinita: ciò significa che **non possono essere estese** a meno che non siano dichiarate con la keyword **open**. Questo approccio promuove l'immutabilità e la sicurezza del codice, evitando estensioni accidentali.

Per abilitare l'ereditarietà:

- si usa **open class** per rendere una classe estendibile;
- i metodi devono anch'essi essere marcati come **open** per poter essere sovrascritti;
- per ridefinire un metodo si usa **override**.

Kotlin supporta la **singola ereditarietà** per le classi, ma consente la **ereditarietà multipla tramite interfacce**. Le interfacce possono avere implementazioni predefinite dei metodi.

2.8 Generics e varianti

Kotlin supporta i generics in modo molto simile a Java, ma con alcune semplificazioni sintattiche e miglioramenti legati alla sicurezza dei tipi.

- La sintassi **fun <T>** o **class Box<T>** consente la definizione di funzioni e classi generiche.
- Per controllare la variabilità (come i wildcard di Java), Kotlin usa le parole chiave:
 - **out T: covarianza** – il tipo può essere usato solo in output (produrre valori);
 - **in T: contravarianza** – il tipo può essere usato solo in input (accettare valori).
- La **where** clause permette di specificare vincoli multipli sui tipi:

```
fun <T> copyIfValid(source: List<T>, target: MutableList<T>)
    where T : CharSequence, T : Comparable<T> { ... }
```

2.9 Funzioni Scoped

Le funzioni scoped in Kotlin – **let**, **run**, **with**, **apply**, **also** – consentono di lavorare su oggetti in modo più leggibile e conciso. Si usano spesso per evitare ripetizioni del nome dell'oggetto o per delimitare blocchi di codice.

- **let**: usa **it** come nome implicito dell'oggetto, utile per chaining o operazioni su nullable.
- **run**: restituisce il risultato dell'ultimo statement, **this** è implicito.
- **with**: simile a **run**, ma riceve l'oggetto come parametro.
- **apply**: ritorna l'oggetto stesso, utile per inizializzazioni fluide.
- **also**: come **apply**, ma usa **it** come oggetto implicito (utile per logging/debug).

Esempio:

```
val user = User().apply {
    name = "Lisa"
    age = 25
}
```

2.10 Collections e Sequences

Kotlin fornisce una ricca API per la gestione delle collezioni, con operazioni funzionali e supporto per valutazione lazy tramite le **Sequences**.

- Factory methods: **listOf**, **mutableListOf**, **setOf**, **mapOf**, ecc.
- Operazioni comuni: **filter**, **map**, **groupBy**, **flatMap**, **fold**, **any**, **all**, **none**, ecc.
- **Sequences** sono collezioni lazy, simili agli Stream in Java 8: consentono trasformazioni su grandi dataset con computazione differita fino al terminal operation.

Esempio:

```
val result = listOf(1, 2, 3, 4)
    .asSequence()
    .filter { it % 2 == 0 }
    .map { it * 2 }
    .toList()
```

2.11 Casting e Smart Casts

Kotlin fornisce strumenti per il casting esplicito e automatico:

- `is` e `!is` per il controllo del tipo:
 `if (x is String) println(x.length) // smart cast`
- `as` per il cast esplicito (potrebbe lanciare un `ClassCastException`).
- `as?` per il cast sicuro: restituisce `null` se il cast fallisce.

Il compilatore effettua automaticamente gli **smart cast** quando verifica che una variabile è già stata controllata per tipo e che non può essere mutata nel frattempo. Questo riduce la necessità di cast espliciti nel codice e migliora la leggibilità.

3 Introduzione a Gradle

Gradle è uno strumento di build e automazione open source, sviluppato in Java, che trova applicazione in molteplici ecosistemi software. Non solo è il sistema di build predefinito per Android, ma supporta anche lo sviluppo per piattaforme JVM (come Java, Kotlin e Groovy), linguaggi nativi (C, C++, Rust, Swift), e persino altri ambienti come Python e Go. Può essere utilizzato sia in versione stand-alone sia integrato in ambienti come IntelliJ IDEA o Android Studio.

3.1 Principi generali

Gradle adotta un approccio basato su convenzioni: assume la presenza di determinati file e directory (come le cartelle per il codice sorgente e per i test), riducendo la necessità di configurazioni esplicite. È progettato per supportare progetti multi-modulo e consente un alto grado di personalizzazione tramite script. Gestisce automaticamente le dipendenze, scaricando i moduli necessari da repository specificati nel progetto.

3.2 Struttura e script di build

Il processo di build è definito attraverso uno o più file di script (tipicamente `build.gradle` in Groovy o `build.gradle.kts` in Kotlin). In questi script vengono specificati:

- le configurazioni del progetto,
- le dipendenze,
- i plugin da utilizzare,
- e le operazioni da automatizzare.

3.3 Inizializzare un progetto Gradle

Le modalità principali per inizializzare un progetto sono:

- Tramite CLI: `gradle init` in una directory creata manualmente.
- Con IntelliJ IDEA: File → New → Project → Gradle.
- Con Android Studio: **la più consigliata** per progetti Android.
- Online: <https://gradle-initializr.cleverapps.io> consente di generare progetti da scaricare.

3.4 Struttura di un progetto Gradle

Un progetto base contiene:

- `build.gradle.kts` – script di build principale.
- `settings.gradle.kts` – configura i moduli del progetto.
- `gradlew`, `gradlew.bat` – wrapper per eseguire gradle.
- `gradle/wrapper` – codice necessario per eseguire gradle localmente.

3.5 Multi-modulo

Android Studio inizializza automaticamente un progetto multi-modulo per supportare architetture complesse. Ogni modulo rappresenta un sottoprogetto che produce un artefatto (ad esempio, un file `.apk` o `.jar`). Ogni modulo ha:

- una propria directory `src`,
- un proprio file `build.gradle.kts`,
- dipendenze e plugin specifici.

3.6 File di configurazione

3.6.1 Parent build.gradle

Contiene le versioni condivise delle librerie e i plugin principali, con l'opzione `apply false` per evitare applicazioni automatiche nei moduli.

3.6.2 Modulo build.gradle

Contiene:

- plugin come `com.android.application` o `com.android.library`,
- sezione `android` con namespace e SDK,
- sezione `dependencies` con riferimenti ad altri moduli o librerie esterne.

3.6.3 settings.gradle.kts

Qui vengono inclusi i moduli che fanno parte del progetto:

```
rootProject.name = "My Application"
include ':app'
include ':mylibrary'
```

3.7 SDK Android

- `compileSdk`: versione API usata per compilare.
- `targetSdk`: versione target su cui testare.
- `minSdk`: versione minima richiesta per eseguire l'app.

L'uso di versioni aggiornate migliora sicurezza e funzionalità, ma riduce la compatibilità con dispositivi più vecchi.

3.8 Tasks

Gradle opera tramite **tasks**, cioè unità operative eseguibili dalla CLI o tramite interfaccia IDE. Le principali includono:

- `build`, `clean`, `assemble`, `test`
- `dependencies`, `signingReport`, `androidDependencies`
- `javadoc`, `wrapper`, `help`, `check`

Eseguibili tramite: `./gradlew <task>`

3.9 Plugin Gradle

I plugin estendono le capacità di Gradle. Possono essere ufficiali o della community. Esempi:

- **Core**: `java`, `java-library`, `application`, `maven-publish`, `checkstyle`, `jacoco`
- **Community**: `kotlin("jvm")`, `kotlin("kapt")`, `com.android.application`

3.10 Dipendenze e Repositories

- Le librerie esterne si specificano nel blocco `dependencies`.
- I luoghi da cui scaricarle si definiscono nel blocco `repositories`.
- Supportati: `mavenCentral()`, `google()`, `jcenter()`, repository locali e aziendali.

3.10.1 Dipendenze transitiva

Gradle gestisce anche le dipendenze delle dipendenze. Comando utile: `gradle -q dependencies --configuration implementation`

3.10.2 Configurazione

Una dipendenza è identificata da:

- nome del gruppo,
- nome dell'artefatto,
- versione.

Esempio: `implementation("org.hibernate:hibernate-core:3.6.7")`

4 Introduzione

Android è una piattaforma open source pensata per i dispositivi mobili, sviluppata per offrire un ambiente completo per lo sviluppo di applicazioni. Include strumenti di sviluppo gratuiti, API per l'accesso all'hardware e supporta diversi linguaggi di programmazione: principalmente Kotlin e Java, ma anche C++ e Rust. Tutte le applicazioni utilizzano lo stesso SDK e condividono le stesse possibilità di interazione con l'hardware, senza distinzioni tra quelle sviluppate da Google o da terze parti.

4.1 L'Open Handset Alliance (OHA)

Fondata nel 2007, l'OHA è una coalizione di 84 aziende leader nel settore tecnologico e mobile (tra cui Google, produttori di chip, operatori mobili e aziende software) che collabora per definire standard aperti per la telefonia mobile e promuovere lo sviluppo di Android.

4.2 Architettura di Android

L'architettura di Android è stratificata, composta da:

- un kernel Linux (v. 5.4 o 4.19) che funge da livello di astrazione hardware (HAL);
- librerie native per grafica, media, sicurezza, database (OpenGL ES, SQLite, WebKit, etc.);
- il runtime Android basato su ART (o Dalvik), una VM ottimizzata per dispositivi mobili;
- un insieme di framework per lo sviluppo di app e componenti software;
- un layer di applicazioni base (browser, email, contatti) facilmente estendibili.

Ogni applicazione Android gira in un processo indipendente e in una propria istanza di VM, isolata per sicurezza e stabilità.

4.3 Sicurezza in Android

La sicurezza è garantita da un modello a livelli:

- Ogni applicazione ha un proprio utente nel sistema operativo e uno spazio isolato.
- Le risorse condivise devono essere dichiarate nel manifest.
- L'accesso ai dati sensibili (contatti, posizione, memoria) richiede permessi espliciti.

Se un'app prova a usare risorse non dichiarate, viene bloccata. Se i permessi non sono concessi, l'installazione può fallire.

4.4 Sviluppo di applicazioni Android

Le applicazioni Android sfruttano le API dell'SDK per accedere alle funzionalità hardware e di sistema. Non vi sono limitazioni tra app di sistema e di terze parti, a patto che dispongano dei permessi corretti. Le app si compongono di codice e dati, installati in uno spazio privato sul file system, ed eseguiti nel contesto utente assegnato in fase di installazione.

4.5 Struttura delle applicazioni

Le applicazioni Android sono suddivise in componenti:

- **Activities:** componenti con interfaccia grafica.
- **Services:** processi in background senza UI.
- **Content Providers:** componenti per la gestione e condivisione dei dati.
- **Broadcast Receivers:** ricevono eventi di sistema o app.

Questi componenti sono dichiarati nel file `AndroidManifest.xml`, che agisce come contratto tra l'app e il sistema. Non esiste un punto di ingresso unico: ogni componente può essere attivato separatamente da un evento del sistema o da un Intent.

4.6 Dettagli sui componenti

4.6.1 Activity

Un'Activity è una singola schermata con UI. Ad esempio, un'app di posta elettronica può avere diverse Activities per la lista messaggi, lettura e composizione. Con lo sviluppo moderno, è comune usare poche Activities e molte Fragments oppure un'unica Surface con UI disegnata dichiarativamente.

4.6.2 Service

Un Service svolge operazioni lunghe in background, come riprodurre musica o raccogliere dati di rete. Non ha interfaccia e deriva da `android.app.Service`.

4.6.3 Content Provider

Gestisce dati strutturati, accessibili anche da altre app tramite ContentResolver. Implementa metodi standard ed estende `android.content.ContentProvider`.

4.6.4 Broadcast Receiver

Ascolta messaggi dal sistema o da altre app (es. batteria scarica, trasferimento completato). Non ha UI, ma può attivare notifiche. Estende `android.content.BroadcastReceiver`.

4.7 Ciclo di vita delle applicazioni

4.7.1 Manifest e avvio

Il manifest definisce componenti, permessi e configurazioni. Quando un evento esterno richiede un'app, Android crea un nuovo processo (possibilmente riutilizzando Zygote), inizializza un oggetto Application e istanzia i componenti richiesti.

4.7.2 Oggetto Application

L'oggetto `android.app.Application` viene creato all'avvio e notifica lo stato dell'app tramite metodi come `onCreate()`, `onConfigurationChanged()`, `onLowMemory()`, `onTerminate()`.

4.7.3 Persistenza

L'applicazione resta in memoria finché almeno un componente è attivo. L'oggetto Application può essere subclassato per mantenere stato globale, ma nelle app moderne si preferisce il pattern Singleton.

4.8 Avvio di un'applicazione

1. L'utente tocca l'icona di un'app.
2. Il processo di sistema Zygote viene clonato (fork).
3. Inizia un messaggio loop.
4. Viene creato un oggetto Application e viene chiamato `onCreate()`.
5. L'activity principale è istanziata e riceve `onCreate()`.
6. L'interfaccia utente viene costruita e mostrata.

4.9 Interazione tra applicazioni

Le app possono comunicare tramite **Intent**: messaggi che definiscono un'azione da eseguire e opzionalmente un set di dati. Il sistema identifica e attiva i componenti in grado di gestire quell'intent, permettendo interazioni modulari e dinamiche senza dipendenze dirette tra classi.

5 Componenti delle App Android

Un'applicazione Android è composta da una o più componenti dichiarate nel file `AndroidManifest.xml`. Le principali categorie sono:

- **Activities:** interfacce grafiche con cui l'utente interagisce.
- **Services:** componenti senza UI, attivi in background.
- **Broadcast Receivers:** ricevono messaggi di sistema o da app.
- **Content Providers:** espongono dati ad altre app.

I componenti vengono creati dal sistema operativo in risposta a richieste rappresentate tramite oggetti `Intent`.

5.1 La classe Activity

`android.app.Activity` è la classe base che fornisce una GUI. Le classi concrete derivano da essa per implementare funzionalità specifiche, e vengono registrate nel manifest. Il sistema ne gestisce il ciclo di vita tramite metodi specifici come `onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()`, `onDestroy()`.

5.1.1 Responsabilità di un'Activity

Un'activity si occupa di:

- Gestire l'interazione con l'utente (GUI, eventi, input).
- Acquisire e rilasciare risorse (sensori, contenuti, ecc.).
- Costruire l'interfaccia grafica.
- Rispondere alle notifiche del ciclo di vita.

5.2 Ciclo di vita e Stack delle Activity

Le activity vengono gestite tramite uno stack: ogni nuova activity viene posizionata in cima e diventa visibile/interattiva. Quando un'activity viene interrotta (ad es. da una chiamata), viene notificata con `onPause()` e/o `onStop()`.

Il ciclo di vita prevede fasi legate a visibilità e interattività:

- `onCreate()` / `onDestroy()` – l'activity esiste ma non è visibile.
- `onStart()` / `onStop()` – visibile ma non interattiva.
- `onResume()` / `onPause()` – visibile e interattiva.

5.2.1 Gestione corretta del ciclo di vita

I metodi devono richiamare quelli della superclasse (es. `super.onCreate()`) per garantire un comportamento corretto. `onPause()` deve essere breve per evitare ritardi nell'avvio della nuova activity.

5.2.2 Salvataggio dello stato

Quando il sistema termina un'activity, invoca `onSaveInstanceState(b: Bundle)` per salvare dati persistenti. In fase di riavvio, i dati sono disponibili in `onCreate()` o `onRestoreInstanceState()`.

5.3 Preparazione dell'interfaccia grafica

5.4 Approccio imperativo

Si crea una gerarchia di oggetti View (es. `LinearLayout`, `TextView`, `Button`) via codice e si collega con `setContentView(...)`.

5.4.1 Approccio dichiarativo con XML

La struttura della UI viene definita in un file XML all'interno di `res/layout`. Gli elementi sono identificati con attributi `id`, utilizzabili in Kotlin via `findViewById`. Vantaggi: manutenibilità, internazionalizzazione, editor visuale.

5.4.2 Jetpack Compose

Nuovo approccio dichiarativo ispirato a React. Le UI sono descritte con funzioni annotate con `@Composable`. Più idiomático in Kotlin, integrato con Material Design e meno dipendente da tool grafici.

5.5 Intents

Un `Intent` è un messaggio che descrive un'azione. Può essere:

- **Esplicito**: indica la classe da attivare direttamente.
- **Implicito**: descrive l'azione da compiere, e il sistema individua l'app adatta.

Gli intent possono contenere dati extra (in un `Bundle`) serializzabili o parcelizzabili.

5.6 Transizioni tra Activity

- `startActivity()` lancia una nuova activity.
- `finish()` termina l'activity corrente.
- Le transizioni possono essere:
 - semplici (una nuova activity si sovrappone);
 - sostitutive (la nuova sostituisce la precedente);
 - con risultato: una activity restituisce un dato alla precedente tramite `ActivityResultLauncher`.

5.7 Broadcast degli Intent

Un'app può inviare un broadcast con `sendBroadcast()`, ricevuto da `BroadcastReceiver` registrati via `registerReceiver()` (post Android 8.0). I receiver devono essere disattivati con `unregisterReceiver()`.

5.8 La classe Context

`android.content.Context` è la superclasse comune di componenti come Activity, Service, Application. Permette:

- accesso a risorse (`getResources()`),
- gestione di file e preferenze (`getSharedPreferences()`),
- avvio di altri componenti (`startActivity()`, `sendBroadcast()`, ...),
- registrazione dei receiver, richiesta servizi di sistema, accesso a layout, ecc.

Ogni componente attivo ha un proprio context, ma esiste anche un **application context** globale.

6 Introduzione a Jetpack Compose

Jetpack Compose è una libreria moderna per la costruzione di interfacce utente in Android. Si tratta di un extbftoolkit dichiarativo, ispirato a React, Flutter e SwiftUI, che consente di descrivere l'interfaccia tramite extttt@Composable functions.

Questo paradigma permette di concentrarsi su extbfcosa vogliamo che l'interfaccia mostri in funzione dello stato attuale, lasciando al framework il compito di gestire gli aggiornamenti (recomposition). Non è più responsabilità dello sviluppatore aggiornare manualmente i widget in seguito a cambiamenti di stato.

6.1 Principi fondamentali

- Le UI sono costituite da funzioni annotate con `@Composable`, che emettono blocchi grafici.
- Le composable functions possono invocare altre composable functions, creando una struttura ad albero.
- Ogni modifica dello stato genera una nuova invocazione delle composable coinvolte.
- Il framework è responsabile della propagazione dei cambiamenti e dell'aggiornamento del layout.

6.2 Sintassi e DSL

Compose sfrutta una DSL basata su funzioni di trailing lambda. Le funzioni composable possono contenere altre UI, permettendo gerarchie nidificate, ad esempio:

```
Column(Modifier.background(Color.Yellow)) {  
    Text("Hello", fontSize = 32.sp)  
    Text("Compose!", fontSize = 32.sp)  
}
```

Ogni componente accetta parametri come `modifier`, che definisce comportamento e stile.

6.3 Elementi base

- `Text(...)`: mostra testo. Parametri: `fontSize`, `color`, `style`, `modifier`.
- `Image(...)`: mostra immagini, usa `Painter`. Richiede `contentDescription`.
- `Canvas(...)`: superficie personalizzabile per grafica disegnata manualmente.
- `Card(...)`: contenitore con elevazione, forma e risposta a clic.

6.4 Layout base

- `Column(...)`: organizza i figli in verticale.
- `Row(...)`: organizza in orizzontale.
- `Box(...)`: sovrappone gli elementi.
- Layout nidificati consentono strutture complesse e reattive.

`Modifier.weight(...)` permette di distribuire lo spazio tra figli.

6.5 Liste e griglie

- `LazyColumn`, `LazyRow`: liste scrollabili che renderizzano solo gli elementi visibili.
- `LazyVerticalGrid`, `LazyHorizontalGrid`: griglie scrollabili.
- DSL: `items`, `itemsIndexed`, `stickyHeader`.

6.6 Modifiers

I modifier permettono di alterare aspetto e comportamento:

- `background(...)`: colore di sfondo.
- `padding(...)`: margini interni.
- `fillMaxSize()`, `clip()`, `offset()`, ecc.
- Possono essere concatenati con `.then(...)`.

Custom modifiers possono essere creati estendendo `Modifier` con nuove funzioni.

6.7 Gestione dello stato

- Le composabile possono essere **stateless** o **stateful**.
- `remember(...)` memorizza il valore attraverso le recomposition.
- `mutableStateOf(...)` incapsula dati reattivi.
- `val (value, setValue) = remember mutableStateOf(...)`
- Le modifiche allo stato generano la recomposition dei componenti che ne dipendono.

6.8 Hoisting e flusso dati

- **State hoisting**: lo stato viene sollevato a composabile più in alto nella gerarchia.
- I componenti figli ricevono lo stato e i setter come parametri.
- `Compose` promuove un **flusso dati unidirezionale**, che va dallo stato alla UI e viceversa attraverso eventi.

6.9 Stilizzazione con Material Theme

- Le UI possono essere stilizzate tramite `MaterialTheme(...)`.
- Permette personalizzazione di colori, tipografia e forme.
- Supporta temi chiaro/scuro.

7 Concurrency e threading in Android

La programmazione concorrente è essenziale quando:

- si vuole sfruttare al massimo le CPU multi-core;
- si desidera evitare di bloccare l'esecuzione durante operazioni di I/O.

Il concetto di **thread** risolve questi problemi: è un flusso indipendente di esecuzione schedulato dal sistema operativo all'interno di un processo. Ogni app parte con un singolo thread (detto "main thread" o "UI thread"), ma può crearne altri.

7.1 Thread in Kotlin

- I thread si creano con `Thread { ... }.start()` oppure tramite la funzione `thread(...)`.
- Ogni thread ha uno stack dedicato per variabili locali e indirizzi di ritorno.
- L'accesso a variabili condivise richiede **sincronizzazione** tramite primitive (`synchronized`, `ReentrantLock`, ecc.).

7.1.1 Problemi comuni

- Tutti i thread condividono lo stesso heap: se usano la stessa referenza, condividono l'oggetto.
- Accessi concorrenti non sincronizzati generano comportamenti inaspettati.
- I thread non si possono cancellare facilmente: serve cooperazione usando `interrupt()`.

7.2 Multithreading in Android

All'avvio, Android crea il processo e il main thread, che:

- gestisce eventi e componenti,
- mantiene una coda di messaggi (`MessageQueue`) gestita in un ciclo infinito (`Looper`).

7.2.1 Looper e MessageQueue

- Il `Looper` estrae messaggi dalla coda e li invia al destinatario.
- La coda può ricevere messaggi anche da altri thread.
- L'accesso concorrente è sincronizzato: un conflitto può causare crash ("Application not responding").

7.3 Thread di sistema e JVM

Android crea altri thread gestiti internamente:

- JVM: GC, JDWP, compilatori JIT, `Finalizer`.
- Sistema: `Binder`, `Signal catcher`, `GL updater`, `hwUITask`.

7.4 Astrazioni di programmazione

- Java: `Thread`, `Runnable`, `ExecutorService`, `Atomic`, `CountDownLatch`, ecc.
- Android: `Looper`, `Handler`, `HandlerThread`.
- Kotlin: **Coroutines**, che semplificano la gestione concorrente.

7.5 Uso di Handler e Looper

`Handler` è una classe che permette l'inserimento di messaggi o `Runnable` in una `MessageQueue`, gestita da un `Looper`. Può eseguire:

- `post(r), postDelayed(r, t)`
- `sendMessage(m), sendMessageDelayed(m, t)`

Il metodo `handleMessage(m)` gestisce ogni messaggio ricevuto.

7.6 HandlerThread

`HandlerThread` è un thread con un `Looper` incorporato. Esempio:

```
val ht = HandlerThread("MyThread")
ht.start()
val h = Handler(ht.looper) { m ->
    // gestisci messaggi qui
    true
}
```

7.7 Problemi tipici e best practice

- Solo il main thread può accedere alla gerarchia delle view: ogni accesso da altri thread causa eccezioni.
- Non è sufficiente salvare un valore in un attributo: bisogna notificare il main thread.
- Si possono usare: `runOnUiThread`, `view.post`, `Handler` creati nel main thread.

7.8 Producer-consumer pattern

Uno schema comune è:

- Producer crea messaggi con dati;
- li inserisce in una `MessageQueue` con `Handler`;
- il consumer (thread con `Looper`) li estrae e li processa con `handleMessage()`.

8 Introduzione alle Coroutines

Le **coroutines** di Kotlin forniscono un'astrazione di alto livello per scrivere codice asincrono e concorrente, evitando il classico "callback hell". A differenza dei thread, sono leggere, scalabili e facilmente gestibili. Il loro funzionamento si basa sul concetto di **suspend function** e sul supporto del compilatore.

8.1 Cos'è una Coroutine

Formalmente, una coroutine è una computazione sospendibile: può essere interrotta in determinati punti e ripresa successivamente anche su un thread diverso. Questo abilita modelli cooperativi di programmazione asincrona, come generatori, future, canali e flussi di dati (Flow).

8.2 Motivazione e vantaggi

Rispetto all'approccio thread-based:

- le coroutines non bloccano il thread ma lo liberano durante operazioni sospendibili;
- consentono un uso efficiente delle risorse (stack, scheduling);
- offrono codice più leggibile e manutenibile, seguendo una logica sequenziale.

8.3 Suspend function e CoroutineScope

Una funzione **suspend** può essere sospesa e ripresa. Può chiamare funzioni normali, ma non può essere chiamata da esse. Le coroutines vivono all'interno di un **CoroutineScope**, che fornisce il **CoroutineContext**.

8.4 Creazione di Coroutines

- **launch {}**: avvia una coroutine senza bloccare, ritorna un oggetto **Job**.
- **async {}**: avvia una coroutine che ritorna un **Deferred<T>** da **await()**.
- **runBlocking {}**: blocca il thread finché le coroutines interne non terminano (per testing).

8.5 Structured Concurrency

Kotlin gestisce le relazioni genitore/figlio tra coroutine:

- i figli ereditano il contesto del genitore;
- il genitore attende la terminazione dei figli;
- un'eccezione in un figlio cancella anche il genitore (a meno di **SupervisorJob**).

8.6 Dispatcher e Context

Ogni **CoroutineScope** ha un **CoroutineContext** che definisce:

- il **Dispatcher** (Main, IO, Default, Unconfined);
- il **Job**;
- l'eventuale **ExceptionHandler** e **CoroutineName**.

È possibile specificare il dispatcher tramite **withContext(...)**.

8.7 Exception e Cancellation

- **launch** non rilancia le eccezioni a meno che non ci sia un handler;
- **async** le rilancia in **await()**;

- tutte le funzioni `suspend` sono cancellabili: per cooperare si usa `isActive`, `ensureActive()`, `yield()`.

8.8 Sincronizzazione e concorrenza

Per gestire accessi concorrenti:

- **Mutex**: sezione critica mutuale;
- **Semaphore**: limite di concorrenza simultanea.

Usare dispatcher a thread singolo può semplificare la concorrenza.

8.9 Flows

`Flow<T>` è un'astrazione per stream asincroni:

- supporta emissione sequenziale di valori nel tempo;
- **cold flow**: non produce dati se non raccolto;
- **hot flow**: emette anche senza listener.

8.9.1 Operazioni intermedie e terminali

- Intermedie: `map`, `filter`, `flatMapMerge`, `onEach`, ...
- Terminali: `collect()`, `toList()`, `reduce()`, `first()`, ...

8.10 Flows vs Sequence

Entrambi sono lazy, ma:

- le sequence sono sincrone, i flow asincroni;
- i flow possono essere sospesi, cancellati, spostati tra thread;
- sequenze lavorano su stack, i flow usano coroutine.

9 Architettura delle applicazioni

Un'applicazione è un software che gestisce dati secondo delle regole di business. L'interfaccia utente è una rappresentazione (spesso derivata) dello stato interno dell'app. Quando l'utente interagisce con l'app (o in seguito allo scorrere del tempo), i dati vengono aggiornati e l'interfaccia si adegua di conseguenza.

Quando la mole di dati è ridotta e le regole semplici, l'implementazione è diretta. Ma con l'aumentare della complessità, serve un'architettura ben definita per controllare l'esplosione della complessità.

9.1 Pattern architetturali

I pattern architetturali sono soluzioni riutilizzabili a problemi comuni nella progettazione del software. Possono rispondere a:

- problemi software (manutenibilità, testabilità);
- problemi operativi (scalabilità, disponibilità);
- problemi di business (costi, time-to-market).

Nel tempo ne sono emersi diversi, adattandosi a contesti in evoluzione o colmando limiti di approcci precedenti.

9.2 Architettura delle app Android

Le app Android sono composte da componenti (Activity, Service, Content Provider, Broadcast Receiver) gestiti dal sistema operativo. Possono essere attivati in ordine non prevedibile e distrutti in qualsiasi momento. Questo rende difficile la progettazione e il testing.

9.3 Separation of Concerns

- Nessun componente dovrebbe contenere dati persistenti nello stato interno, dato che potrebbe essere distrutto senza preavviso.
- Nessun componente dovrebbe dipendere dall'esistenza attiva di un altro (salvo contratto esplicito).
- Le classi Activity/Fragment devono occuparsi solo di UI e interazione con l'OS.
- Il contenuto della UI deve riflettere un modello, preferibilmente persistente.

9.4 Lo stato dell'applicazione

- UI state (es. menù aperto o chiuso);
- Form state (input attuali);
- Application state (dati utente);
- Dati read-only (es. articoli).

9.5 Principali architetture

9.6 MVC - Model View Controller

- **Model:** logica di business e sorgente dei dati.
- **View:** interfaccia grafica e presentazione dei dati.
- **Controller:** riceve gli eventi della View e aggiorna il Model.

Pro: separazione chiara, test del Model facile.

Contro: forte accoppiamento tra componenti.

9.6.1 MVP - Model View Presenter

- Il Presenter interagisce con View e Model.
- Aggiorna la View secondo i cambiamenti del Model.

Pro: decoupling tra View e Model.

Contro: gestione View via codice è onerosa.

9.6.2 MVI - Model View Intent

- **Intent** modellano l'interazione utente.
- **ViewModel** riceve gli Intent e aggiorna lo stato osservato dalla View.
- Stato immutabile, flusso dati unidirezionale.

Pro: flusso prevedibile, separazione netta.

Contro: più complesso e reattivo.

9.6.3 MVVM - Model View ViewModel

- La ViewModel collega Model e View.
- Espone dati osservabili (LiveData, StateFlow).
- La View si aggiorna automaticamente in base allo stato.

Pro: decoupling alto, ottima testabilità.

Contro: curva di apprendimento e complessità iniziale.

9.7 Android e MVVM

Google supporta MVVM con strumenti come ViewModel, LiveData, Room, Retrofit. Le ViewModel sono progettate per sopravvivere ai cambiamenti di configurazione e non devono contenere riferimenti a UI o context.

9.8 ViewModel: caratteristiche

- Gestisce lo stato della UI in modo persistente.
- Comunica col Model tramite metodi dedicati.
- Espone dati osservabili (es. `LiveData<T>`, `State<T>`, `StateFlow<T>`).
- Sopravvive alla rotazione dello schermo.

9.9 Osservabili e Compose

- LiveData si usa bene con View classiche.
- State si integra con Jetpack Compose.
- StateFlow e SharedFlow funzionano con coroutines.

9.10 Esempi ViewModel

```
class CounterViewModel : ViewModel() {  
    private val _count = MutableLiveData<Int>(0)  
    val count: LiveData<Int> = _count  
    fun increment() { _count.value = _count.value!! + 1 }  
}
```

```
class CounterViewModel : ViewModel() {  
    private val _count = mutableStateOf(0)  
    val count: State<Int> = _count
```

```
fun increment() { _count.value++ }  
}
```

9.11 Lifecycle-aware components

- `LifecycleObserver` e `LifecycleEventObserver` gestiscono eventi di ciclo di vita.
- `ProcessLifecycleOwner` permette di osservare lo stato globale dell'app.
- `LiveData` è lifecycle-aware.

9.12 Conclusione

MVVM è l'architettura consigliata da Google per Android moderno. Fornisce una chiara separazione, facilita testing e manutenibilità, ed è supportata nativamente dalle librerie Android Jetpack.

10 Introduzione alla Data Layer

La data layer è la componente dell'applicazione responsabile della logica di business e della gestione dello stato dell'app. È divisa in tre sottosezioni principali:

- **Business Logic** (o Domain Model): implementa la logica dell'app e si basa su sorgenti dati locali e/o remote.
- **Remote Data Source**: si occupa del dialogo con un server remoto, tipicamente via REST.
- **Local Data Source**: gestisce la persistenza locale, solitamente tramite un DBMS (come SQLite).

10.1 Modelli e disaccoppiamento

I modelli sono completamente indipendenti dalle Views e da qualsiasi componente Android. Questo isolamento facilita il testing e la manutenibilità del codice. I dati gestiti dal Model devono avere una rappresentazione neutrale, guidata solo dai requisiti di dominio:

- **DTO** (Data Transfer Object): dati ricevuti dal server.
- **Entity**: dati salvati nel database locale.

10.2 Aggiornamenti asincroni e stato

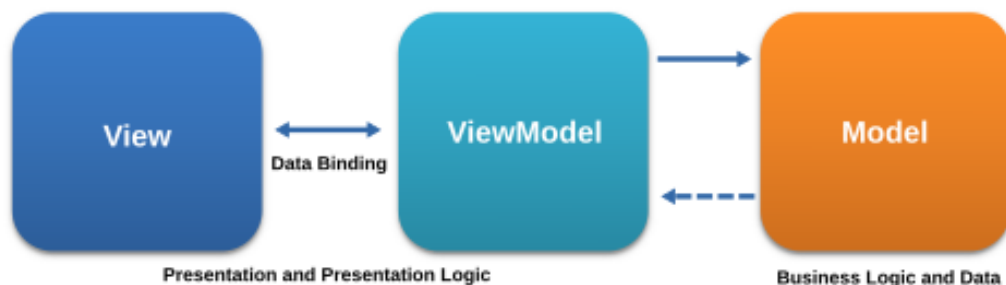
Gli aggiornamenti del modello possono essere asincroni, derivare da eventi locali o remoti, ed entrare in conflitto con il ciclo di vita dei componenti UI. Per reagire a tali cambiamenti, il modello può esporre i propri dati tramite:

- `Flow<T>` di Kotlin,
- Osservabili di RxJava,
- `LiveData<T>`.

10.3 Operazioni lunghe e threading

Le operazioni su data layer possono richiedere tempo e non devono essere eseguite sul thread principale per evitare ANR (Application Not Responding). La gestione del threading è spesso responsabilità della ViewModel, con il supporto delle Kotlin coroutines.

10.4 Architettura del Model



Il Model comunica con la ViewModel attraverso osservabili e riceve dati da:

- Remote source (Retrofit),
- Local source (Room).

10.5 Implementazione del Model

Il modello espone metodi che rappresentano le azioni possibili da parte dell'utente. Può anche occuparsi di trasformazioni sui dati: conversioni, filtri, ordinamenti. Il tipo e la complessità dell'implementazione dipende da come è strutturata la sorgente dati (locale, remota, o entrambe).

10.6 Persistenza locale con Room

Room è una libreria ORM che astrae l'uso diretto di SQLite:

- Usa annotazioni per definire entità (@Entity), DAO (@Dao), e database (@Database).
- Supporta funzioni `suspend` e `Flow<T>` tramite la libreria `room-ktx`.
- Offre supporto a migrazioni di schema con oggetti `Migration`.

10.7 DAO e entità

Un esempio di entità:

```
@Entity(tableName = "users")
data class User(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    @ColumnInfo(name = "first_name") val firstName: String,
    @ColumnInfo(name = "last_name") val lastName: String
)
```

E relativo DAO:

```
@Dao
interface UserDao {
    @Query("SELECT * FROM users")
    fun getAll(): LiveData<List<User>>

    @Insert(onConflict = REPLACE)
    fun save(user: User)

    @Delete
    fun delete(user: User)
}
```

10.8 Database singleton

La classe database è una singleton per garantire un'unica istanza:

```
@Database(entities = [User::class], version = 1)
abstract class UserDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
    companion object {
        @Volatile private var INSTANCE: UserDatabase? = null
        fun getDatabase(context: Context): UserDatabase =
            INSTANCE ?: synchronized(this) {
                INSTANCE ?: Room.databaseBuilder(
                    context.applicationContext,
                    UserDatabase::class.java,
                    "user_database"
                )
            }
    }
}
```



```

        ).build().also { INSTANCE = it }
    }
}

```

10.9 Accesso ai dati

Room supporta:

- `LiveData<T>` e `Flow<T>` per l'osservazione continua dei dati;
- funzioni `suspend` per operazioni asincrone;
- query personalizzate e multi-tabella;
- compatibilità con `Cursor`.

10.10 Accesso remoto con Retrofit

Retrofit è una libreria HTTP che permette di definire servizi con annotazioni:

```

interface APIInterface {
    @POST("/api/users")
    suspend fun createUser(@Body user: User): User

    @GET("/api/users")
    suspend fun getUsers(): List<User>
}

```

Richiede permesso Internet nel manifest e la configurazione tramite `Retrofit.Builder`. Supporta:

- funzioni `suspend`,
- ritorno di `Call<T>`, `LiveData<T>`, `Flow<T>` (con adattatore).

10.11 Retrofit + Flow

È possibile utilizzare Retrofit con Flow per ricevere dati osservabili. Serve un `CallAdapterFactory` specifico come `FlowCallAdapterFactory`.

10.12 Conclusioni

La data layer è cruciale per un'app ben progettata. Room e Retrofit offrono un'integrazione efficace con Kotlin, MVVM e le coroutine, garantendo modularità, riusabilità e reattività.

10.13 Introduzione alla Dependency Injection

Nel paradigma della programmazione orientata agli oggetti, molte classi delegano parte del proprio comportamento ad altre classi. Queste classi delegate prendono il nome di **dipendenze**. È comune che un oggetto abbia bisogno di un'istanza di una dipendenza per poter funzionare correttamente: questa dipendenza viene spesso fornita nel costruttore. Tuttavia, lasciar gestire la creazione delle dipendenze alla stessa classe ne aumenta il coupling e ne riduce la flessibilità.

10.14 Problemi nella gestione delle dipendenze

- Difficoltà di test: le classi fortemente accoppiate non possono essere testate isolatamente.
- Configurazioni duplicate: ogni classe potrebbe dover conoscere dettagli implementativi delle dipendenze.
- Difficoltà di riutilizzo: condividere una stessa dipendenza tra oggetti diversi è complicato.

10.15 Principio di Inversion of Control e DI

La **Dependency Injection** (DI) è un pattern che implementa l'**Inversion of Control**: la responsabilità della creazione degli oggetti viene spostata da chi li usa a un componente esterno chiamato **contenitore DI**.

10.16 Refactoring e interfacce

Per ridurre l'accoppiamento:

- Le classi non devono costruire direttamente le proprie dipendenze.
- Si introducono **interfacce** per astrarre le dipendenze.
- Il costruttore riceve istanze delle interfacce come parametri.

10.17 Esempio

```
class MyRepository(val myDb: MyDatabase, val myApi: MyApi)

interface MyDatabase { ... }
class MyDatabaseImpl(context: Context): RoomDatabase(), MyDatabase { ... }

interface MyApi { ... }
class MyApiImpl(baseUrl: String): MyApi { ... }
```

10.18 Framework per la DI in Android

- **Spring**: contesto centralizzato, gestione a runtime.
- **Dagger2**: generazione statica del codice a compile-time.
- **Koin**: runtime DI completamente scritto in Kotlin.
- **Hilt**: basato su Dagger2, integrato con Android.

10.19 Introduzione a Hilt

Hilt è una libreria per la DI sviluppata da Google. Genera codice a compile-time tramite annotazioni e si integra con Jetpack. Richiede:

- Plugin e dipendenze nel file **build.gradle** (progetto e modulo).
- Annotazioni per identificare classi e punti di iniezione.

10.20 Configurazione

Nel file `build.gradle` (root):

```
buildscript {
    dependencies {
        classpath("com.google.dagger:hilt-android-gradle-plugin:2.48")
    }
}
```

Nel modulo `app`:

```
plugins {
    id("kotlin-kapt")
    id("com.google.dagger.hilt.android")
}
dependencies {
    implementation("com.google.dagger:hilt-android:2.48")
    kapt("com.google.dagger:hilt-compiler:2.48")
}
```

10.21 Annotazioni principali

- `@HiltAndroidApp`: su una sottoclasse di `Application`, abilita Hilt.
- `@AndroidEntryPoint`: su componenti Android (`Activity`, `Fragment`, `Service`).
- `@Inject`: per segnalare dove devono essere iniettate le dipendenze.
- `@Module` e `@Provides`: per fornire dipendenze personalizzate.

10.22 Scope

Gli scope determinano la durata della vita di un'istanza:

- `@Singleton`: istanza unica per tutta l'app.
- `@ActivityRetainedScoped`, `@ActivityScoped`, `@FragmentScoped`, ecc.

10.23 Esempio con Hilt

```
@HiltAndroidApp
class MyHiltApp : Application()

@AndroidEntryPoint
class MainActivity : AppCompatActivity() {
    @Inject lateinit var player: MusicPlayer
}

class MusicPlayer @Inject constructor() { fun play(id: String) { ... } }
```

10.24 Modules con Hilt

```
@Module
@InstallIn(SingletonComponent::class)
object MusicModule {
    @Singleton
    @Provides
    fun provideMusicDB(@ApplicationContext context: Context): MusicDatabase {
```

```
        return Room.databaseBuilder(context, MusicDatabase::class.java, "music.db").build()
    }
}
```

10.25 Integrazione con Jetpack

- ViewModel: `@HiltViewModel`, costruttore con `@Inject`
- Navigation, Compose, WorkManager: supportati da Hilt

11 Introduzione alla Navigazione

Le applicazioni mobili sono composte da più schermate, ognuna delle quali rappresenta un sottoinsieme dell'informazione gestita e abilita diverse interazioni utente. Per spostarsi tra queste schermate, si utilizza un sistema di navigazione basato su **routes**.

La libreria **Jetpack Navigation** supporta questo meccanismo, fornendo componenti per gestire la transizione tra schermate, l'animazione, il deep linking e vari pattern di UI come drawer, tab e bottom navigation bar.

11.1 Dipendenze necessarie

Nel file `build.gradle` è necessario includere:

```
val navVersion = "2.8.1"
implementation("androidx.navigation:navigation-compose:$navVersion")
```

11.2 Componenti principali della Navigation

- **NavController**: gestisce lo stato della navigazione, la back stack, le route e l'accesso ai parametri.
- **NavHost**: composable che visualizza la schermata associata alla route corrente.
- **Navigation Graph**: descrive l'insieme delle route e le relative schermate.

11.3 Definizione delle route

Ogni schermata è identificata da una **route**, una stringa simile a un percorso URL. Può contenere parametri obbligatori (es. `editProject/projectId`) o opzionali (es. `editProject?projectId=...`).

Esempio:

```
NavHost(navController = navCtrl, startDestination = "home") {
    composable("home") { HomeScreen() }
    dialog("addProject") { AddProjectDialog() }
    composable("editProject/{projectId}",
        arguments = listOf(navArgument("projectId") { type = NavType.IntType })
    ) { entry ->
        EditProjectScreen(projectId = entry.arguments?.getInt("projectId"))
    }
}
```

11.4 Navigazione tra schermate

Le azioni di navigazione vengono gestite dal **NavController**:

```
class Actions(val navCont: NavHostController) {
    val addProject: () -> Unit = { navCont.navigate("addProject") }
    val editProject: (Int) -> Unit = { id -> navCont.navigate("editProject/$id") }
    val navigateBack: () -> Unit = { navCont.popBackStack() }
}
```

Queste azioni possono essere passate come parametri ai composable figli.

11.5 Gestione dello stato di navigazione

- Ogni entry nella back stack può contenere un oggetto **SavedStateHandle**.

- Questo permette di salvare valori chiave/valore che persistono anche dopo la terminazione del processo.
- Utile per restituire risultati a schermate precedenti.

11.6 Scaffold e Navigation UI

Scaffold gestisce la struttura visuale di base dell'app, comprendente:

- **TopAppBar**,
- **FloatingActionButton (FAB)**,
- **BottomAppBar** o **NavigationBar**,
- **Drawer** (**NavigationDrawer**).

Esempio:

```
Scaffold(
  topBar = { TopAppBar(...) },
  bottomBar = { BottomAppBar(...) },
  floatingActionButton = { FAB(...) }
) { innerPadding ->
  MyNavHost(padding = innerPadding)
}
```

11.7 Altri componenti di navigazione

- **NavigationBar**: per dispositivi mobili.
- **NavigationRail**: per tablet e display più ampi.
- **NavigationDrawer**: menù laterale scorrevole.
- **Tabs**: per organizzare contenuti in categorie.

12 Input dell'utente nei sistemi Android

In Android l'interazione avviene tramite molteplici canali di input, da tastiere e mouse fino a sensori touch capacitivo avanzati. I gesti multitouch (come il pinch) vengono rilevati e modellati da eventi di tipo `MotionEvent`, introdotti in Android 2.2.

12.0.1 MotionEvent

Ogni gesto è rappresentato da una sequenza di oggetti `MotionEvent`. Tra le principali proprietà troviamo:

- `actionMasked`: azione principale (DOWN, UP, MOVE, POINTER_DOWN...)
- `pointerCount`: numero di dita attive
- `getX(index)`, `getY(index)`, `getPressure(index)`: coordinate e pressione
- `getHistoricalX(...)`: per accedere ai dati di tocco precedenti

12.1 Gestione dei tocchi in Compose

Jetpack Compose offre diversi `modifier` per gestire le interazioni:

- `clickable(...)`: intercetta clic
- `draggable(...)`: rileva trascinamenti
- `pointerInput(...)`: accesso a eventi touch a basso livello

12.1.1 Esempio: tap e long press

```
.pointerInput(Unit) {
    detectTapGestures(
        onTap = {...}, onDoubleTap = {...},
        onLongPress = {...}, onPress = {...}
    )
}
```

12.2 Drag e Swipe

Compose distingue:

- **Scroll**: esplorazione contenuti (`verticalScroll(...)`)
- **Swipe**: azioni irreversibili (`anchoredDraggable(...)`) con stato `AnchoredDraggableState<T>`

12.3 Gestii multitouch

Il `detectTransformGestures(...)` consente zoom, rotazioni e traslazioni. Deve essere preceduto da `Modifier.graphicsLayer(...)` per rendere applicabili le trasformazioni.

12.4 Animazioni: introduzione

Le animazioni rafforzano la comunicazione visiva, migliorano l'usabilità e la percezione della fluidità. Possono essere:

- **Funzionali**: microinterazioni, caricamenti, navigazione.
- **Decorative**: branding, storytelling.

12.5 12 principi dell'animazione secondo Disney

1. **Squash and Stretch**: conferisce elasticità e massa, utile per rendere oggetti flessibili (es. palla rimbalzante).
2. **Anticipation**: prepara lo spettatore a un'azione importante, aumentando la chiarezza e l'impatto.
3. **Staging**: ogni azione deve essere chiara e comprensibile grazie a posa, inquadratura e timing.
4. **Straight Ahead e Pose to Pose**: due approcci complementari alla costruzione del movimento: spontaneo vs controllato.
5. **Follow Through e Overlapping Action**: il movimento continua anche dopo che l'azione principale è conclusa, e le parti del corpo si muovono in tempi diversi.
6. **Slow In e Slow Out**: transizioni più naturali con accelerazioni e decelerazioni fluide.
7. **Arcs**: i movimenti organici seguono archi, non linee rette.
8. **Secondary Action**: azioni minori che arricchiscono e supportano l'azione principale (es. un sorriso mentre si parla).
9. **Timing**: controlla la velocità e l'enfasi, influisce sul peso e sulla personalità dell'azione.
10. **Exaggeration**: esagerare il movimento per enfatizzare l'intento narrativo o emozionale.
11. **Solid Drawing**: competenza nel disegno tridimensionale, equilibrio e prospettiva coerente.
12. **Appeal**: il personaggio o l'elemento animato deve essere esteticamente interessante e coinvolgente.

12.6 Animazioni in Compose

12.6.1 Cambi di gerarchia

In Jetpack Compose, i cambi di gerarchia si riferiscono all'aggiunta, rimozione o sostituzione di interi blocchi di UI (Composable) in risposta a modifiche dello stato. Compose offre alcuni strumenti potenti per animare queste transizioni in modo fluido e intuitivo:

- **AnimatedVisibility**:

Permette di animare la comparsa e scomparsa di un Composable in base a una condizione booleana. È ideale per elementi che devono “entrare” o “uscire” dalla scena con animazioni (es. dissolvenza, slide, espansione).

```
AnimatedVisibility(visible = isExpanded) {  
    Text("Contenuto visibile")  
}
```

È possibile personalizzare le animazioni di entrata (enter) e uscita (exit) specificando effetti come `fadeIn`, `slideInVertically`, `expandHorizontally`...

- **Crossfade**:

Serve per sostituire un contenuto con un altro attraverso un'animazione a dissolvenza. Molto utile quando si vuole transizionare tra due stati mutuamente esclusivi (es. una schermata “loading” e una schermata con i dati caricati).

- **AnimatedContent**:

È un composable avanzato che combina le funzionalità di `Crossfade` e `AnimatedVisibility`, permettendo una transizione animata tra due contenuti che possono anche avere una struttura interna diversa. A differenza di `Crossfade`, consente anche animazioni bidirezionali complesse e personalizzate in base al tipo di contenuto.

12.6.2 Animazioni di attributi

Quando parliamo di animazioni di attributi, ci riferiamo a tutte quelle animazioni che non cambiano la struttura della UI (cioè non aggiungono o rimuovono elementi), ma modificano un

valore numerico nel tempo: ad esempio, opacità, dimensioni, colore, posizione, ecc.

- **Animatable<T>** è una classe che permette di animare manualmente un valore numerico (es. `Float`, `Color`...) usando coroutine. Si usa insieme a `LaunchedEffect(...)` per eseguire l'animazione quando uno specifico valore cambia.

```
val offset = remember { Animatable(0f) }
LaunchedEffect(target) {
    offset.animateTo(
        targetValue = 100f,
        animationSpec = tween(durationMillis = 500)
    )
}
```

- **animate*AsState(...)**: interpolazioni reattive.

Questa è la forma più semplice e dichiarativa per creare un'animazione fluida quando un valore cambia reattivamente. L'asterisco (*) è un segnaposto:

- `animateFloatAsState(...)`
- `animateDpAsState(...)`
- `animateColorAsState(...)`

Quando il valore associato cambia (es. `isVisible = true` → `false`), Compose animerà automaticamente la transizione al nuovo stato, interpolando i valori nel tempo.

```
val alpha by animateFloatAsState(
    targetValue = if (isVisible) 1f else 0f,
    animationSpec = tween(durationMillis = 300)
)
```

- **AnimationSpec** Queste classi definiscono come avviene l'animazione nel tempo. Alcuni esempi: `tween`, `spring`, `keyframes`, `repeatable`, `infiniteRepeatable`, `snap`

12.6.3 Easing

Le curve di easing definiscono come un valore cambia nel tempo durante un'animazione, ovvero modulano la velocità con cui l'interpolazione avviene tra il valore iniziale e quello finale. In termini pratici, determinano se l'animazione:

- parte lentamente e poi accelera (`ease-in`),
- parte veloce e rallenta alla fine (`ease-out`),
- oppure ha una velocità variabile con accelerazioni e decelerazioni (`ease-in-out`).

Tecnicamente, un easing è una funzione che trasforma l'intervallo temporale normale `[0, 1]` in un nuovo intervallo `[0, 1]`, alterando così la progressione del tempo percepito durante l'animazione.

13 Introduzione a Firestore

Cloud Firestore è un database NoSQL orientato ai documenti, parte della piattaforma Firebase di Google. Offre un backend scalabile e in tempo reale, adatto sia ad applicazioni semplici che complesse. È accessibile tramite diverse piattaforme, tra cui Android, iOS, Web e REST API.

13.1 Modello NoSQL e Architettura

- I dati sono organizzati in **collezioni** e **documenti**.
- Ogni documento è un oggetto JSON con chiavi e valori (numeri, stringhe, booleani, timestamp, array, mappe, riferimenti).
- Le collezioni contengono documenti, che a loro volta possono contenere sub-collezioni.
- I documenti sono limitati a 1MB di dimensione e possono avere fino a 20.000 campi.

13.2 Querying e Sincronizzazione

- Firestore permette interrogazioni complesse con filtri e ordinamenti.
- Le query sono ottimizzate tramite indici e possono essere eseguite in tempo reale o una sola volta.
- I dati possono essere sincronizzati automaticamente con il dispositivo.
- È possibile accedere anche offline grazie alla cache automatica.

13.3 Modellazione dei dati

- **Nested fields** e **sub-collezioni** permettono di strutturare dati gerarchici.
- Le sub-collezioni sono preferibili a campi annidati quando si ha molta informazione da strutturare.
- Ogni documento ha un identificativo univoco all'interno della collezione.

13.4 Relazioni tra dati

- Firestore non supporta join tra collezioni.
- Le relazioni si modellano con riferimenti o con la denormalizzazione dei dati (ridondanza).
- In caso di riferimenti, la cancellazione non è propagata automaticamente.

13.5 Accesso da Android

- La libreria Firebase Firestore consente accesso e aggiornamento dei dati tramite oggetti di tipo `Firestore`.
- `val db = FirebaseFirestore.getInstance()`
- I riferimenti a documenti e collezioni si creano con:

```
val coll = db.collection("courses")  
val doc = db.collection("courses").document("mad")
```

13.6 Scrittura e Lettura dei dati

13.7 Scrittura

```
db.collection("courses").document("mad2025")  
    .set(modelData())  
    .addOnSuccessListener { ... }  
    .addOnFailureListener { ... }
```

13.7.1 Lettura

```
db.collection("courses").document("mad2025")
    .get()
    .addOnSuccessListener { res ->
        val course = res.toObject(Course::class.java)
    }
```

13.8 Kotlin e serializzazione

- Si possono usare mappe o data class Kotlin.
- Le annotazioni `@PropertyName` possono personalizzare la mappatura campo-json.

13.9 Realtime updates e Flow

- Con `addSnapshotListener` si ascoltano modifiche in tempo reale.
- È possibile convertire questi callback in Flows con `callbackFlow` di Kotlin.
- `awaitClose` permette di gestire la cancellazione del listener.

13.10 Query avanzate

- Operatori: `whereEqualTo`, `whereIn`, `arrayContains`, ...
- Limitazioni: no range su campi diversi, no `!=`, supporto limitato per OR.
- Ordinamento: `orderBy`, `limit`, `startAt`, `endBefore`, ...

13.11 MVVM e Firestore

- Il Model funge da proxy per Firestore.
- La ViewModel usa `StateFlow` per propagare i dati alla UI.
- Pattern compatibile con Compose.

13.12 Sicurezza e regole di accesso

- Le regole sono definite con un linguaggio DSL nel pannello Firebase.
- Esempi:

```
match /users/{userId} {
    allow read, update: if request.auth.uid == userId;
    allow create: if request.auth.uid != null;
}
```
- Si possono specificare permessi granulari: `read`, `write`, `get`, `list`, `create`, `update`, `delete`.

13.13 Operazioni atomiche

- **Transaction**: blocco atomico di operazioni lettura/scrittura, con `rollback`.
- **BatchedWrite**: insieme di scritture atomiche.
- Operazioni pronte: `increment`, `arrayUnion`, `arrayRemove`.

13.14 Persistenza offline e indexing

- Firestore memorizza i dati usati anche offline (abilitata di default).
- Gli indici migliorano le prestazioni delle query complesse.
- I composite index si gestiscono da console o CLI.

14 Introduzione ai Services

Un **Service** è un componente Android che esegue operazioni in background, senza offrire un'interfaccia utente. Può essere utilizzato da altri componenti dell'app o anche da altri processi. Esistono due principali categorie di servizi:

- **Started Service**: avviato da un componente tramite `startService(...)`.
- **Bound Service**: legato a un componente tramite `bindService(...)`.

14.1 Started Services

Un servizio è considerato "started" quando viene avviato con `startService(...)`. Dopo l'avvio:

- Può continuare a funzionare anche se il componente chiamante viene distrutto.
- In genere esegue un'operazione singola (es. download di un file) e poi si ferma con `stopSelf()`.
- Per essere eseguito a lungo termine, deve mostrarsi all'utente come servizio in foreground con una notifica.

14.2 Bound Services

Un servizio è "bound" quando un componente si connette ad esso con `bindService(...)`, creando un'interfaccia client-server. Le sue caratteristiche:

- Consente richieste e risposte interattive tramite un oggetto `IBinder`.
- Può essere condiviso tra più componenti contemporaneamente.
- Viene distrutto automaticamente quando tutti i componenti associati si disconnettono.

14.3 Gestione del ciclo di vita dei servizi

- `onCreate()`: chiamato alla creazione del servizio.
- `onStartCommand()`: esegue l'operazione richiesta (started service).
- `onBind()`: restituisce l'interfaccia per il bound service.
- `onDestroy()`: pulizia finale prima della distruzione.

14.4 Implementare un servizio personalizzato

Un servizio può essere esteso dalla classe **Service**. Si può usare un `HandlerThread` per eseguire operazioni intensive in un thread separato, mantenendo responsiva l'app.

14.4.1 onStartCommand e ritorni

`onStartCommand(...)` deve restituire un valore intero:

- `START_NOT_STICKY`: il sistema non ricrea il servizio se viene terminato.
- `START_STICKY`: il sistema lo ricrea ma con `Intent` null.
- `START_REDELIVER_INTENT`: il sistema lo ricrea e reinvia l'ultimo `Intent`.

14.5 Limitazioni e Foreground Services

A partire da Android 8.0:

- I servizi in background hanno limitazioni.
- Solo i **foreground services** possono girare indefinitamente, ma richiedono una `Notification` visibile.

- Le app in background possono usare i servizi solo per pochi minuti dopo l'uscita dallo stato attivo.

14.6 Alternative moderne: **WorkManager**

Per task asincroni e differibili, **WorkManager** è l'alternativa consigliata:

- Gestisce il lavoro anche se l'app o il dispositivo vengono riavviati.
- Supporta vincoli (idle, batteria, rete, ecc.).
- Implementa la logica in una sottoclasse di **Worker**.

14.6.1 Esempio di **Worker**

```
class MyWorker(appCtx: Context, params: WorkerParams): Worker(appCtx, params) {
    override fun doWork(): Result {
        // Logica del task
        return Result.success() // o .failure(), .retry()
    }
}
```

```
val myRequest = OneTimeWorkRequestBuilder<MyWorker>().build()
WorkManager.getInstance(context).enqueue(myRequest)
```

14.7 Bound Services con **IBinder**

Il metodo `onBind()` restituisce un **IBinder** che consente al client di interagire con il servizio.

- Se il client e il servizio sono nello stesso processo, si può usare una classe **Binder** custom.
- Alternativamente, si può usare **Messenger** o **AIDL** per IPC.

14.7.1 Esempio con **Binder**

```
class BoundService : Service() {
    private val binder = MyBinder()
    inner class MyBinder : Binder() {
        fun getService(): BoundService = this@BoundService
    }
    override fun onBind(intent: Intent?): IBinder = binder
    fun customFunction(): String = "Servizio attivo"
}
```

14.8 Dichiarazione nel Manifest

Ogni servizio va dichiarato nel file **AndroidManifest.xml**:

```
<service android:name=".MyService" />
```

15 Panoramica dello sviluppo cross-platform

Lo sviluppo di applicazioni mobili cross-platform si basa sul concetto di ”**Write Once, Run Anywhere**” (WORA), cioè scrivere una sola volta il codice sorgente ed eseguirlo su più piattaforme. Tuttavia, questo slogan non sempre riflette la realtà, poich il testing e il debugging devono essere comunque condotti su ogni piattaforma.

15.1 Motivazioni principali

- **Riduzione dei costi e dei tempi di sviluppo.**
- **Condivisione delle competenze:** meno necessità di sviluppatori esperti su ciascuna piattaforma.
- **Rapid Prototyping** e MVP.
- **Uniformità dell’esperienza utente** su più dispositivi.

15.2 Limitazioni e problematiche

- Si implementano solo funzionalità comuni tra piattaforme.
- Difficoltà nell’integrare codice nativo e librerie esistenti.
- Possibili perdite di performance.
- Dipendenza dal framework utilizzato.
- Necessità di testare su ogni piattaforma.

15.3 Differenze strutturali tra Android e iOS

- **Modello di esecuzione:** Android usa componenti in ”liquid processes”, iOS processi monolitici.
- **Comunicazione inter-processo:** Binder vs ApplicationExtension.
- **Gestione memoria:** GC (Android) vs ARC (iOS).
- **SDK:** linguaggi diversi (Java/Kotlin vs Swift/Objective-C), layout e eventi gestiti in modo non identico.

15.4 Framework e tecnologie principali

15.5 WebView-based

- **Apache Cordova:** struttura l’app come una WebView potenziata da plugin.
- **Ionic:** framework Angular-based con supporto a Capacitor e Electron.
- **Pro:** conoscenze diffuse, sviluppo rapido.
- **Contro:** performance scarse, UX lontana da quella nativa.

15.5.1 Native-widget-based

- **React Native:** sviluppato da Meta, usa componenti nativi pilotati da un thread JS.
- **.NET MAUI:** evoluzione di Xamarin, usa C e XAML, supporta Android/iOS/macOS/Windows.
- **Pro:** UI nativa, buone performance.
- **Contro:** necessità di bridge, potenziale overhead.

15.5.2 Custom-widget-based

- **Flutter:** sviluppato da Google, usa Dart e Skia per disegnare direttamente i widget.
- **Unity, Godot, Unreal:** nati per il gaming, usano GPU e rendering a basso livello.
- **Pro:** altissime performance, controllo totale sulla UI.
- **Contro:** interfacce non native, curva di apprendimento.

15.6 Kotlin Multiplatform (KMP)

- Permette di condividere logica di business (non la UI) tra Android, iOS, Web, JVM.
- Si integra con framework nativi (Jetpack Compose, SwiftUI) o Compose Multiplatform.
- Strumenti come Koin permettono DI cross-platform.
- <https://kmp.jetbrains.com/>

15.7 Compose Multiplatform

- Estensione di Jetpack Compose per UI multiplatforma.
- Usa widget propri, non nativi.
- Ottimo per condividere UI in progetti Kotlin.
- Repositorio GitHub: <https://github.com/JetBrains/compose-multiplatform>

15.8 Confronto tra tecnologie

	Flutter	React Native	.NET MAUI
Start-up Time	Veloce	Medio	Medio
App Size	Piccola	Media	Media
Hot Reload	Eccellente	Eccellente	Buono
UI Perf.	Eccellente	Buona	Buona
Community	Eccellente	Eccellente	In crescita
Accesso nativo	Eccellente	Buono	Eccellente

15.9 Tendenze emergenti

- PWAs con supporto a WebAssembly (Flutter 4).
- Integrazione AI/LLM nelle app multiplatforma.
- Crescita delle piattaforme Low-Code / No-Code (FlutterFlow, Adalo, ecc.).
- Refactoring e CI/CD avanzati per deploy rapidi.

16 XR: Extended Reality

Extended Reality (XR) è un termine ombrello che comprende: **Virtual Reality (VR)**, **Augmented Reality (AR)** e **Mixed Reality (MR)**. La distinzione tra queste tecnologie si basa sul livello di immersione e interazione tra l'utente e il contenuto digitale.

16.0.1 Virtual Reality (VR)

Simulazione immersiva di un ambiente realistico che coinvolge più canali sensoriali. Si distingue in:

- **Desktop VR**: esperienza via monitor.
- **Immersive VR**: con dispositivi HMD, CAVE, tracciamento.

I concetti fondamentali sono: **immersion**, **presence** e **interaction**.

16.0.2 Immersione e Presenza

- **Immersione**: quanto il sistema stimola i canali sensoriali umani (vista, udito, tatto...). Dipende da HW/SW.
- **Presenza**: sensazione soggettiva di "essere là". Dipende da immersione e interazione.

16.1 AR e MR

- **AR mobile**: tramite smartphone.
- **AR see-through**: visori ottici/video.
- **MR**: interazione avanzata e integrazione tra reale e virtuale.
- **SLAM**: localizzazione e mappatura simultanee per ancorare contenuti nello spazio.

16.2 XR System: QOC e Interaction

Domande fondamentali (QOC): come visualizzare, cosa tracciare, come manipolare, collaborare, spostarsi. Tecnologie di interazione: **touch**, **hand-tracking**, **voice**, **gestures**, **controllers**, **eye-gaze**.

16.3 Tecniche di Manipolazione

- **Egocentriche**: virtual hand, Go-Go, raycasting.
- **Exocentriche**: World-in-miniature, Scaled-world grab.

16.4 Locomotion in VR

Classificata in **Magica** (teletrasporto, joystick), **Mundana** (camminata fisica), **Body-centric**, **Vehicle-centric**, **Redirected walking**. <https://locomotionvault.github.io/>

16.5 Dispositivi XR

- **Tethered VR**: collegati a PC (es. HTC Vive).
- **Standalone VR**: all-in-one, no PC (es. Meta Quest).
- **MR**: Apple Vision Pro, Meta Quest Pro, HoloLens 2.

16.6 Display e UI in XR

- **Diegetic**: nella narrazione (es. orologio nel gioco).
- **Non-diegetic**: overlay classico (HUD).

- **Meta:** feedback indiretti (es. schermo rosso se colpiti).
- **Spatial:** elementi 3D nello spazio (es. guida luminosa).

16.7 AndroidXR: OS e Visione

AndroidXR è la nuova iniziativa Google per unificare XR su Android:

- Un OS XR standardizzato per visori.
- App 2D eseguibili in ambienti XR (Home Space).
- Supporto ad app XR avanzate (Full Space).
- Jetpack Compose XR e Jetpack SceneCore.
- Compatibilità con Unity e WebXR.

16.8 Jetpack Compose XR: SpatialUI

- Subspace, SpatialPanel, Orbiter, Volume, SpatialDialog, SpatialPopup
- App distribuite nello spazio 3D (FSM)
- Composable con layout 3D (SpatialRow, SpatialColumn)
- Adaptive UI e transizione HSM → FSM

16.9 Jetpack SceneCore

API più potente per realtà aumentata:

- SLAM, ARCore, plane detection, ray casting.
- Scene Graph con Entity-Component System (ECS).
- Supporto a modelli GTLF, interazioni personalizzate, ambienti custom.

16.10 HSM vs FSM

- **HSM (Home Space Mode):** ambienti 2D standardizzati. No spatial UI.
- **FSM (Full Space Mode):** ambienti XR completi con UI 3D, spatial audio, ambienti personalizzati.

16.11 Conclusione

AndroidXR apre a un ecosistema XR più integrato, sfruttando librerie esistenti e introducendo nuovi strumenti (Jetpack Compose XR, SceneCore). Le app XR possono evolvere da UI 2D verso esperienze pienamente immersive, sfruttando spatial panel, passthrough, 3D models e gesture.